

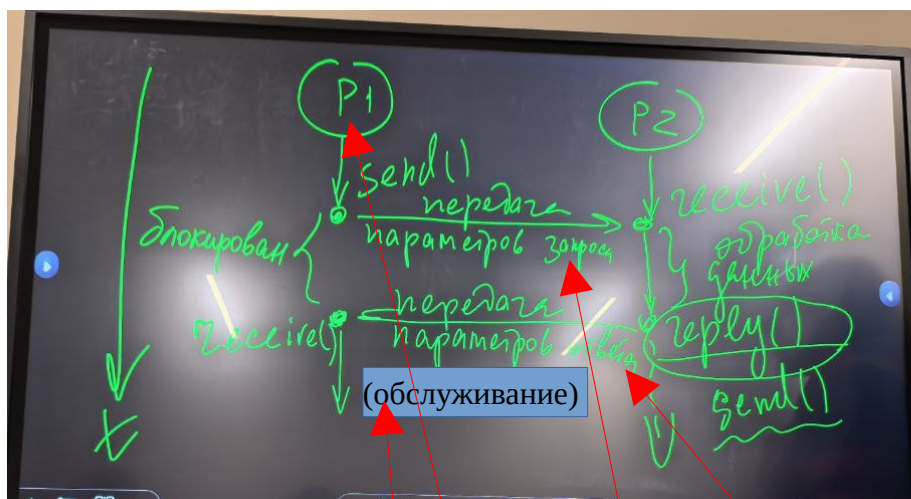
На лекции говорили, что наиболее общим способом взаимодействия процессов является обмен сообщениями. Наиболее общим в том смысле, что обмен сообщениями может выполняться параллельными процессами на отдельно стоящей машине, то есть на мейнфрейме, и в распределенной системе, то есть в сети.

Для обмена сообщениями любая ОС предоставляет соответствующие API типа `send` и `receive`.

Давайте рассмотрим фактически временную диаграмму взаимодействия двух процессов. Процесс `p1` и процесс `p2`, причем процесс `p1` заинтересован в обработке его запроса процессом `p2`. Какое-то время процесс `p1` выполняется, и когда ему нужно выполнить запрос к процессу `p2` он вызывает `send()`. В результате `send()` осуществляется передача параметров запроса (ну фактически можно дописать: для обслуживания). Процесс `p2`, чтобы получить запрос на своей стороне должен вызвать `receive()`.

Какое-то время процесс обрабатывает данные.

Для того, чтоб отличать процесс приема данных и отправки сообщения, указывается ключевое слово `reply`. Соответственно, это передача параметров ответа. Причем, все время до получения ответа, процесс `p1` блокирован. Что делает процесс `p2` после того, как обслужил процесс `p1`? Получив ответ процесс `p1` продолжает свое выполнение. Как уже я сказала, это **своеобразная временная диаграмма**, т. е. Мы даже можем обозначить ось `t` — время.



Почему ключевое слово `reply` абсолютно не обязательно? То есть здесь `send()`, а вот у исходного примера, на который я ориентируюсь, человек забыл написать `receive()`. То есть для обмена сообщениями на самом деле достаточно двух системных вызовов `send()` и `receive()`. Ну, как мы увидим скоро, в Linux: `send message()` и `receive message()` - специальные функции.

Давайте теперь посмотрим, какие возможны ситуации.

Процесс `p2` дошел до точки отправки запроса. Вот слово **запрос**. Вы видите **ответ**, но именно важно также помнить, что `p2` обслужил `p1`. Почему обращаю на это ваше внимание? Потому что это в общем-то взаимодействие «клиент-сервер». Один процесс заинтересован в том, чтобы другой процесс выполнил какую-то работу, и заинтересован в получении результата этой работы. Вот здесь пишу **обслуживание**, просто потому что потом мы с вами с этим обслуживанием будем все время иметь дело. Вроде и слово «сервер» есть, но вытянуть из вас слово обслуживание практически невозможно. Этот пришел в точку запроса, а `p2` задержался, он не пришел в точку получения, значит `p1` будет блокирован при отправке `send()`. Отправляет он параметры запроса, но это `send()` - отправка.

Может быть другая ситуация. `p2` пришел в точку приема, а `p1` в точку отправки не пришел, тогда `p2` будет блокирован при приеме.

Также p2 может быть блокирован при ответе, но в данной схеме это невозможно, потому что p1 не выполняется синхронное взаимодействие, процесс p1 блокирован, но в общем случае такая ситуация возможна, когда p1 не блокирован, когда реализовано асинхронное взаимодействие.

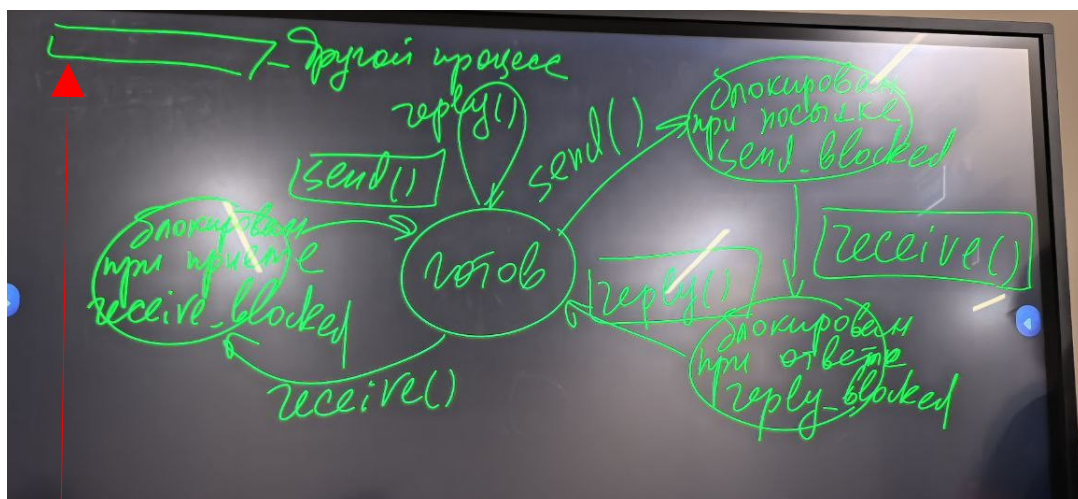
В связи с этим, при обмене сообщениями выделяются **3 состояния блокировки процесса при передаче сообщения**

Диаграмма. Она будет на экзамене. Это базовая диаграмма именно взаимодействия параллельных процессов. У вас будет на лекции тема Микроядерная архитектура. Там эта диаграмма имеет огромное значение.

Также хочу сказать, что ключевое слово `reply` на самом деле не выдуманно. Такой API с таким названием имеется в операционной системе QNX (RTOS - Real-Time Operating System). QNX, как я понимаю, очень популярна в нашей стране, потому что эта ОС условно-платная. Для обучения ее можно использовать бесплатно. Если под эту ОС вы разрабатываете коммерческие продукты, вы обязаны купить лицензию.

Если вы наберете в поисковике Real-Time Operating System, вы увидите очень широкий перечень самых разных ОС. Фирма Windows так же разрабатывает RTOS. Это современная тенденция — предоставить в распоряжение разработчиков вот такую ОС.

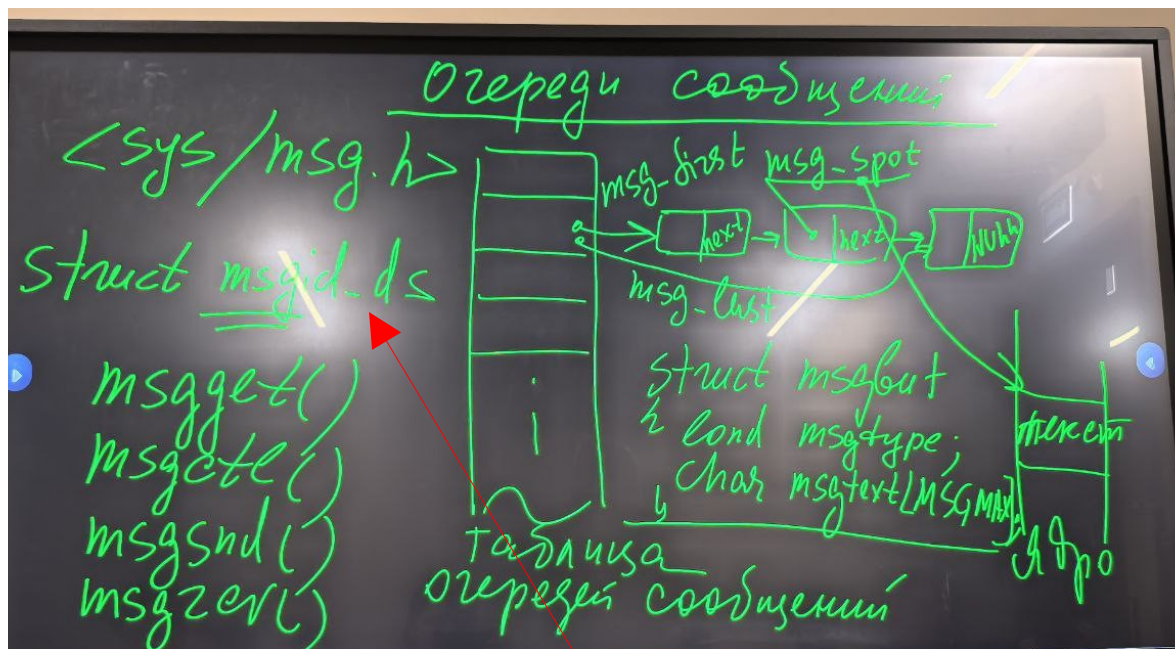
### Диаграмма Три состояния блокировки процесса при передаче сообщений



В **рамочку** обведен другой процесс, т. е. API, которые выполняет другой процесс.

UNIX system files имеет такое средство взаимодействия параллельных процессов как очереди сообщений.

### Очереди сообщений



Соответственно в ядре UNIX имеется таблица очередей сообщений. Библиотека C `<sys/msg.h>`. msg — сокращение от «message».

Обращаю ваше внимание, что все названо единообразно в ядре. Они это сделали, чтобы проще было запоминать, и впоследствии работать с этим.

**Отступление:** Вот я много раз обращаю внимание. И у Дейтла, и я все время говорю. Один из вас написал readlock и readunlock, а ведь написано **startread** и **stopread**. И на лекции я так диктовала. Это никак не lock и unlock, это start и stop. Я заставляю переделывать.

Вот. Меняется только приставка: semid\_ds, shmid\_ds, msgid\_ds. Каждый элемент этой таблицы (напоминаю вам, что таблица — это массив структур) описывает одну очередь сообщений. И это в полном смысле очередь, каждый элемент которой имеет указатель next. Ну это уже внутренняя организация в ядре. При этом, раз это очередь, два указателя msg\_first и msg\_last, и поле data это msg\_spot. И этот spot — это указатель на область данных ядра, в которой находится текст сообщения.

На очереди сообщений определены следующие **системные вызовы**:

- msgget()
- msgctl()
- msgsnd()
- msgrcv()

На сообщениях определена структура, в которой есть поле msgtype и строка с данными сообщения struct msgbuf.

Давайте посмотрим manual: man msgsnd...

Когда процесс передает сообщение в очередь сообщений, ядро создает для него новую запись и помещает ее в конец связного списка записей, соответствующих сообщениям указанной очереди.

В каждой такой записи указываются:

- тип сообщения;
- число байт данных, содержащихся в сообщении (а как проще сказать? Длина сообщения);
- указатель на другую область данных ядра, где фактически находится сообщение (msg\_spot).

Ядро копирует сообщение из адресного пространства процесса отправителя в область данных ядра, и процесс-отправитель может завершиться. Не только завершиться, он может продолжить свое выполнение, но вот указывается, что может завершиться как предельный вариант. А сообщение останется доступным для чтения другими процессами. Значит что исключается, какая блокировка? Ну вот смотрите в диаграмму состояний блокировки процесса. Ответ: При отправке — send.

Когда какой-то процесс выбирает сообщение из очереди, ядро копирует его в адресное пространство процесса-получателя сообщения. **Процесс может выбрать сообщение тремя способами:**

1. Процесс может взять самое старое сообщение в очереди. Что это значит? Старое — с чем связано слово? Ответ: со временем. То есть раньше всех. Всё связано со временем. Первым пришел — это значит, что пришел раньше всех. А кто в хвосте очереди, пришел позже всех. Имеет значение, откуда вы пробегаете очередь: если с головы — там будет самое старое, а в хвосте самое последнее.

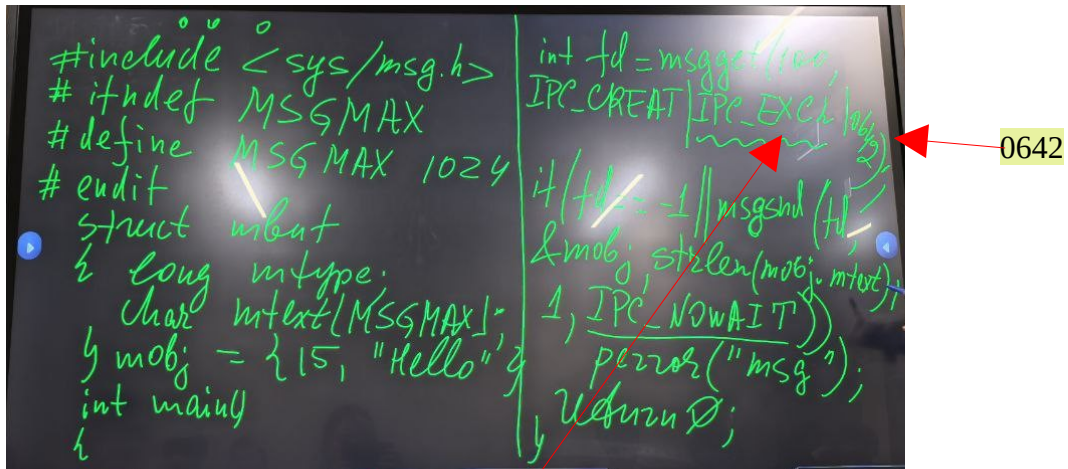
2. Процесс может взять сообщение, если его идентификатор совпадает с идентификатором, который указал процесс. Это очень важная фраза. Это формализация того, как процесс может взять сообщение. Иногда такая формализация необходима в каких-то отчетах, в статьях. Это в чистом виде формализация. Если существуют несколько сообщений с таким идентификатором, то берется самое старое. Опять же, если вы программисты, то имеет значение, откуда вы пробегаете очередь: если с головы, то первый подходящий, если с хвоста, то последний.

3. Процесс может выбрать сообщение, числовое значение типа которого является наименьшим из меньших или равным значению типа, который указал процесс. Если существуют несколько сообщений, удовлетворяющих этому условию, то берется самое старое. Что в этом случае исключается, какая блокировка? Ответ: receive — при получении.

И это как раз и есть демонстрация особенностей вот этого средства взаимодействия, которое они называли «очереди сообщений». Но при этом, может вы заметили, что такое взаимодействие процессов не гарантирует получение процессом конкретных данных. То есть это очень специфическое средство взаимодействия, крайне специфическое. Но с его помощью можно исключить блокировки процессов при отправке и приеме сообщений.

**Блокировки — это ЗЛО.** В каком смысле? Ответ: задержка выполнения. Но зло часто неизбежное. От него пытаются уйти, избавиться, но часто это невозможно.

ПРИМЕР: я напишу только одну библиотеку.



Хочу обратить ваше внимание вот на этот флаг `IPC_EXCL`. В принципе, его надо ставить везде, где вызывается `get`: и там, где `semget()`, и там, где `shmget()`. Это флаг `EXCL` — exist control, то есть этот флаг предписывает системе особенно тщательно проверять, существует ли данный объект. Особенно этот флаг постоянно используется с системным вызовом `open()`. Как вы должны знать системный вызов `open()` может открыть существующий файл, а может создать новый. Вот когда вы вызываете флаг `IPC_CREAT`, то обязательно надо указывать флаг `IPC_EXCL`. Это гарантирует, что вы не потеряете информацию, уже записанную в файл.

Для `get` — `IPC_CREAT`, `IPC_EXCL`

Для `send` — `IPC_NOWAIT` (для `get` это смысла не имеет).

Процесс отправил сообщение «Hello» в очередь и завершился. Это сообщение останется доступным другим процессам. Для получения сообщения они должны знать идентификатор очереди. Здесь также используется функция `ftok()`, о котором мы говорили ранее, для идентификации.

В мануале в примерах она и используется.

Мы с вами рассмотрели три очень разных, существенно отличающихся друг от друга, три средства передачи данных между параллельными процессами. Какие? Ответ: `pipe`, `shared memory`, очереди сообщений (`message queue`).

Они очень отличаются. Каждое средство обладает уникальными особенностями:

1. На `pipe` реализована потоковая модель передачи данных. Кроме того, на `pipe` реализовано средство взаимного исключения, и передача данных симплексная.
2. Разделяемая память — это буфер, можно сказать, обычный буфер. У него нет никаких средств взаимного исключения, обычно используется с семафорами. В отличие от `pipe` информация, записанная в разделяемый сегмент, может быть прочитана многократно, и при чтении этой информации она никуда не девается, может быть прочитана многократно. Если она не будет затерта какой-то другой информацией, она так и будет там находиться до тех

пор, пока существует этот разделяемый сегмент. Но, как мы видели, мы использовали разделяемые сегменты и в производстве-потреблении, и в читателях-писателях, соответственно, блокировки обеспечиваются не самим разделяемым сегментом, а средствами взаимoisключения, которые управляют доступом к разделяемому сегменту, в частности, семафорами.

3. А очереди сообщений вообще ни на что не похожи. И, как мы только что с вами разобрали, они как раз могут исключать блокировки, но при этом гарантии того, что процесс получит именно ту информацию, которая ему нужна, нет. Очень специфическое средство. Но, как вы обратили, наверное, внимание, все три средства взаимодействия создаются в ядре системы, поскольку это средства взаимодействия параллельных процессов. Ключевое слово — процесс. Нельзя путать процесс и поток. Основное отличие — **потоки собственного адресного пространства не имеют и выполняются в адресном пространстве процесса.**

В силу того, что они разделяют адресное пространство одного процесса, они могут разделять глобальные переменные.